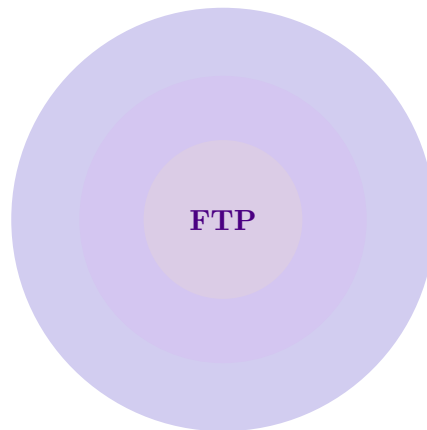


TP FTP

Systèmes et Réseaux

Compte rendu du projet de transfert de fichiers

Nous décrivons ici le code réel du dépôt, son protocole réseau, son architecture maître / esclaves et les fonctionnalités Q1 à Q17



Réalisé par

A1kaaaaa
CASSOU Baptiste

Année universitaire : 2025–2026
8 octobre 2026

Table des matières

1	Introduction	3
2	Vue d'ensemble	4
2.1	Idée générale	4
2.2	Bilan question par question	4
2.3	Ce que le code fait aujourd'hui	4
3	Organisation du dépôt	5
3.1	Binaires principaux	5
3.2	Répertoires importants	5
3.3	Fichiers les plus importants	5
4	Architecture générale	6
4.1	Rôle de chaque programme	6
4.1.1	Le client	6
4.1.2	Le maître	6
4.1.3	Les esclaves	6
4.2	Démarrage du cluster	6
4.3	Démarrage d'une session client	7
5	Le protocole réseau	8
5.1	Pourquoi nous utilisons TCP	8
5.2	Messages principaux	8
5.2.1	La requête client	8
5.2.2	La réponse serveur	8
5.2.3	Les messages de contrôle	9
5.3	Ordre des octets et sérialisation	9
5.4	Lecture exacte et écriture exacte	9
5.5	Sécurité minimale sur les noms de fichiers	10
6	Comment une session fonctionne	11
6.1	Boucle générale	11
6.2	Commande <code>get</code>	11
6.2.1	Côté client	11
6.2.2	Côté serveur	11
6.2.3	Pourquoi la reprise marche	11
6.3	Commande <code>ls</code>	12
6.4	Commande <code>put</code>	12
6.5	Commande <code>rm</code>	12
6.6	Commande <code>auth</code>	12
6.7	Commande <code>bye</code>	13
7	Répartition de charge et réplication	14
7.1	Enregistrement des esclaves	14
7.2	Carte du cluster	14
7.3	Redirection du client	14
7.4	Reconnexion après panne	14
7.5	Réplication de <code>put</code> et <code>rm</code>	14
7.5.1	Pour <code>put</code>	15
7.5.2	Pour <code>rm</code>	15

8	Explication par étapes du sujet	16
8.1	Étape 1 : base du service FTP	16
8.1.1	Q1 et Q2	16
8.1.2	Q3 et Q4	16
8.1.3	Q5	16
8.1.4	Q6 et Q7	16
8.2	Étape 2 : blocs, session longue et reprise	16
8.2.1	Q8	16
8.2.2	Q9	17
8.2.3	Q10	17
8.3	Étape 3 : maître et esclaves	17
8.3.1	Q11	17
8.3.2	Q12	17
8.3.3	Q13	17
8.3.4	Q14	17
8.4	Étape 4 : commandes avancées	18
8.4.1	Q15	18
8.4.2	Q16	18
8.4.3	Q17	18
9	Fonctions importantes	19
9.1	Fonctions côté client	19
9.2	Fonctions côté serveur	19
9.3	Fonctions du protocole et du transfert	20
9.4	Wrappers système utiles	20
10	Tests	21
10.1	Scripts présents dans le dépôt	21
10.2	Ce que ces scripts vérifient	21
10.3	Méthodes de vérification	21
10.4	Commandes utiles	21
11	Limites actuelles	22
12	Conclusion	23

1 Introduction

Nous avons réalisé un petit service de transfert de fichiers inspiré de FTP. Nous ne cherchons pas à reproduire tout le protocole FTP historique. Nous avons créé un protocole plus petit, plus simple, et adapté au sujet.

Dans ce projet, nous voulions surtout comprendre quatre idées :

- comment un client et un serveur parlent sur TCP ;
- comment on définit un protocole applicatif avec des messages fixes ;
- comment on transfère un fichier sans accès direct au disque distant ;
- comment on ajoute peu à peu des fonctions plus avancées.

La version actuelle du dépôt couvre les questions **Q1 à Q17**. Nous avons donc :

- un client interactif ;
- un maître sur le port 2121 ;
- deux serveurs esclaves ;
- les commandes `get`, `put`, `ls`, `rm`, `auth` et `bye` ;
- la reprise d'un `get` interrompu ;
- la reconnexion du client en cas de panne pendant un `get` ;
- la réplication simple de `put` et `rm` entre esclaves ;
- une sérialisation explicite des messages réseau avec prise en compte de l'endianness.

Ce compte rendu a été réécrit à partir du code actuel. Nous décrivons donc le fonctionnement réel du dépôt, et pas seulement l'intention du projet.

2 Vue d'ensemble

2.1 Idée générale

Le système contient trois programmes :

- **clientFTP** : le programme lancé par l'utilisateur ;
- **masterFTP** : le maître, qui reçoit les clients sur 2121 puis les redirige ;
- **serverFTP** : un esclave, qui traite réellement les commandes FTP.

Le maître ne transfère pas de fichiers. Il sert seulement à choisir un esclave. Le vrai travail se fait sur un esclave.

2.2 Bilan question par question

Question	État	Ce que nous avons fait
Q1	Réalisée	Nous avons défini le type <code>typereq_t</code> pour les commandes du protocole.
Q2	Réalisée	Nous avons défini la structure <code>request_t</code> pour les messages client vers serveur.
Q3	Réalisée	Nous avons créé les binaires client et serveur, ainsi que l'écoute réseau.
Q4	Réalisée	Nous avons géré l'arrêt propre avec <code>SIGINT</code> .
Q5	Réalisée	Nous avons séparé le répertoire client et le répertoire serveur.
Q6	Réalisée	Nous avons reçu une requête, validé la demande et envoyé une réponse.
Q7	Réalisée	Nous avons ajouté la commande interactive <code>get</code> côté client avec statistiques.
Q8	Réalisée	Nous avons envoyé les fichiers par blocs de taille fixe.
Q9	Réalisée	Nous avons gardé une session ouverte pour plusieurs commandes successives.
Q10	Réalisée	Nous avons ajouté la reprise d'un téléchargement et la robustesse après coupure brutale.
Q11	Réalisée	Nous avons fixé un nombre d'esclaves et des ports dédiés.
Q12	Réalisée	Nous avons relié le maître et les esclaves au démarrage.
Q13	Réalisée	Nous avons redirigé le client du maître vers un esclave.
Q14	Réalisée	Nous avons reconnecté automatiquement le client pendant un <code>get</code> .
Q15	Réalisée	Nous avons ajouté la commande <code>ls</code> .
Q16	Réalisée	Nous avons ajouté <code>put</code> et <code>rm</code> , avec réplication simple.
Q17	Réalisée	Nous avons protégé <code>put</code> et <code>rm</code> avec <code>auth</code> .

2.3 Ce que le code fait aujourd'hui

La boucle normale est la suivante :

1. nous démarrons les esclaves ;
2. nous démarrons le maître ;
3. le maître enregistre les esclaves ;
4. le client se connecte au maître ;
5. le maître envoie l'adresse d'un esclave ;
6. le client ouvre une vraie session FTP avec cet esclave ;
7. le client envoie plusieurs commandes jusqu'à `bye`.

3 Organisation du dépôt

3.1 Binaires principaux

Binaire	Source principale	Rôle
bin/clientFTP	src/clientFTP.c	Lit les commandes utilisateur, se connecte au maître ou à un esclave, puis affiche les résultats.
bin/masterFTP	src/server_master.c	Enregistre les esclaves au démarrage et redirige les clients en round-robin.
bin/serverFTP	src/serverFTP.c	Traite les commandes FTP sur un esclave et gère aussi la réplication sur le port de contrôle.

3.2 Répertoires importants

Répertoire	Utilité
include/	Tous les fichiers d'en-tête, les constantes, les structures et les prototypes.
src/	Le code C du projet.
data_client/	Les fichiers locaux du client. Les téléchargements arrivent ici. Les fichiers envoyés avec <code>put</code> partent de là.
data_server/	Les fichiers vus par les serveurs. Les <code>get</code> , <code>ls</code> , <code>put</code> et <code>rm</code> agissent ici.
scripts/	Les scripts de démonstration et de test.
logs/	Les logs générés par les scripts.
bin/	Les objets compilés et les exécutables.

Attention

Dans ce dépôt, les deux esclaves travaillent dans le même dossier `data_server/`. La séparation entre client et serveur est donc bien réelle, mais la séparation entre les deux esclaves reste surtout logique.

Autrement dit, nous avons bien un protocole de réplication dans le code, mais les deux esclaves ne stockent pas leurs fichiers dans deux dossiers physiques différents.

3.3 Fichiers les plus importants

Fichier	Rôle
include/ftp_shared.h	Définit les constantes, les types de requêtes, les statuts et les structures réseau.
src/ftp_protocol.c	Séréalise, déséréalise, valide et envoie les messages du protocole.
src/ftp_transfer.c	Envoie et reçoit les payloads des fichiers et des listes.
src/clientFTP.c	Gère l'entrée utilisateur et la connexion côté client.
src/client_requests.c	Implémente <code>get</code> , <code>put</code> , <code>ls</code> , <code>rm</code> , <code>auth</code> , <code>bye</code> côté client.
src/serverFTP.c	Démontre un esclave, lance les workers et garde le port de contrôle ouvert.
src/server_requests.c	Implémente les commandes côté serveur et la réplication.
src/server_master.c	Démontre le maître, enregistre les esclaves et répartit les clients.
src/csapp.c	Fournit des wrappers utiles pour les sockets, les E/S robustes et les signaux.
Makefile	Construit les trois binaires et propose quelques cibles pratiques.

4 Architecture générale

4.1 Rôle de chaque programme

4.1.1 Le client

Le client fait trois choses :

- il lit une ligne tapée par l'utilisateur ;
- il transforme cette ligne en commande ;
- il envoie une requête réseau puis affiche le résultat.

Le client sait parler soit directement à un esclave, soit au maître. Si nous lui donnons le port 2121, il demande d'abord une redirection au maître. Sinon, il se connecte directement au port donné.

4.1.2 Le maître

Le maître sert de répartiteur de charge très simple.

Il fait deux choses :

- au démarrage, il contacte tous les esclaves sur leurs ports de contrôle ;
- ensuite, il choisit un esclave pour chaque nouveau client.

Le choix se fait en **round-robin**. Cela veut dire que nous prenons un esclave, puis le suivant, puis le suivant, et ainsi de suite.

4.1.3 Les esclaves

Chaque esclave fait en réalité deux travaux en même temps :

- il traite les commandes FTP des clients sur son port client ;
- il écoute aussi les messages de réplication sur son port de contrôle.

Le père garde le port de contrôle. Les workers traitent les clients.

4.2 Démarrage du cluster

Le démarrage suit toujours le même ordre :

1. nous lançons un esclave avec un identifiant, par exemple 1 ;
2. cet esclave se place dans `data_server/` ;
3. il ouvre son port client et son port de contrôle ;
4. il attend que le maître se connecte sur le port de contrôle ;
5. nous faisons la même chose avec le second esclave ;
6. nous lançons ensuite le maître ;
7. le maître se connecte aux ports de contrôle 4001 et 4002 sur `127.0.0.1` ;
8. chaque esclave envoie son identité au maître ;
9. le maître construit la carte du cluster et la renvoie à chaque esclave ;
10. seulement après cela, le maître écoute sur 2121.

Dans la configuration actuelle :

- l'esclave 1 écoute pour les clients sur 3001 et pour le contrôle sur 4001 ;
- l'esclave 2 écoute pour les clients sur 3002 et pour le contrôle sur 4002 ;
- le maître écoute pour les clients sur 2121.

4.3 Démarrage d'une session client

Quand nous lançons `clientFTP`, nous faisons d'abord :

1. un changement de répertoire vers `data_client/` ;
2. une connexion réseau ;
3. l'affichage de l'invite FTP `>>`.

Ensuite, deux cas existent :

- si nous passons un port d'esclave comme 3001, le client se connecte directement à cet esclave ;
- si nous passons 2121, le client se connecte d'abord au maître, reçoit l'adresse d'un esclave, ferme la première connexion, puis ouvre une seconde connexion vers cet esclave.

5 Le protocole réseau

5.1 Pourquoi nous utilisons TCP

Nous utilisons TCP parce que nous voulons un flux fiable d'octets.

Pour ce projet, TCP nous apporte :

- l'ordre des données ;
- la détection des coupures ;
- une base simple pour envoyer à la fois des messages de protocole et des fichiers.

Idée

TCP n'envoie pas des “messages” tout prêts. TCP envoie un flux. Cela veut dire qu'un `read()` peut renvoyer moins d'octets que prévu. C'est pour cela que nous utilisons des fonctions robustes comme `rio_readn()` et `rio_writen()`.

5.2 Messages principaux

Nous avons défini les messages communs dans `include/ftp_shared.h`.

5.2.1 La requête client

La structure `request_t` contient :

- la version du protocole ;
- le type de requête ;
- un offset pour la reprise ;
- une taille de fichier pour `put` ;
- une taille de bloc ;
- un nom de fichier ;
- un login ;
- un mot de passe.

5.2.2 La réponse serveur

La structure `response_t` contient :

- un statut ;
- le type de requête concerné ;
- la taille totale du fichier ;
- l'offset accepté par le serveur ;
- la taille d'un payload supplémentaire.

Le champ `payload_size` est utile pour `ls`, car la réponse contient alors un texte après l'en-tête.

5.2.3 Les messages de contrôle

Nous utilisons aussi d'autres messages :

- `slave_hello_t` : identité d'un esclave ;
- `slave_cluster_t` : carte complète du cluster ;
- `ftp_replication_request_t` : ordre de réplication pour `put` et `rm` ;
- `ftp_control_reply_t` : accusé de réception sur le canal de contrôle.

5.3 Ordre des octets et sérialisation

Un point important du projet est maintenant le suivant :

Attention

Nous n'envoyons plus les structures C brutes sur le réseau.
Nous écrivons chaque champ nous-mêmes dans un buffer réseau. Cela évite les problèmes de padding mémoire et les problèmes d'endianness entre machines.

Concrètement :

- pour les ports réseau, les wrappers sockets utilisent déjà `htons()` ;
- pour nos messages applicatifs, nous utilisons `htonl()` et `ntohl()` pour les champs 32 bits ;
- pour les champs 64 bits, nous utilisons un helper dédié du même genre ;
- à la réception, nous faisons l'opération inverse avant d'utiliser les valeurs.

Nous faisons aussi une validation après décodage :

- version du protocole ;
- type de requête ;
- statut ;
- taille de bloc ;
- cohérence des offsets et des tailles ;
- cohérence des messages de contrôle.

Si un message semble avoir été envoyé avec le mauvais ordre des octets, nous le refusons.

Nous ne convertissons pas le contenu des fichiers eux-mêmes. C'est normal : un fichier doit voyager comme une suite brute d'octets. Seuls les en-têtes du protocole doivent être convertis.

5.4 Lecture exacte et écriture exacte

Le protocole fonctionne parce que nous savons toujours combien d'octets lire :

- une requête a une taille fixe sur le réseau ;
- une réponse a une taille fixe sur le réseau ;
- un message de contrôle a une taille fixe sur le réseau ;
- un fichier ou une liste a une taille annoncée dans l'en-tête.

Cela simplifie beaucoup le traitement :

- nous lisons l'en-tête complet ;
- nous interprétons les champs ;
- nous lisons ensuite exactement le payload attendu.

5.5 Sécurité minimale sur les noms de fichiers

La fonction `ftp_is_safe_filename()` refuse les noms dangereux.
Nous bloquons par exemple :

- `..` ;
- `.` ;
- les chemins avec `/` ;
- les noms vides.

Cette règle est simple, mais elle évite les sorties du répertoire de travail.

6 Comment une session fonctionne

6.1 Boucle générale

Une session FTP sur un esclave suit cette idée simple :

1. le client envoie une requête ;
2. le serveur la lit ;
3. le serveur la valide ;
4. le serveur exécute l'action ;
5. le serveur envoie une réponse ;
6. si nécessaire, un payload suit la réponse.

La connexion reste ouverte pour les commandes suivantes. Elle se termine avec **bye** ou en cas d'erreur.

6.2 Commande **get**

6.2.1 Côté client

Quand nous faisons **get fichier**, le client :

1. vérifie le nom du fichier ;
2. regarde si un fichier local du même nom existe déjà ;
3. utilise sa taille locale comme offset de reprise ;
4. construit une requête **GET** ;
5. envoie la requête ;
6. reçoit la réponse ;
7. reçoit le reste du fichier et l'écrit localement ;
8. affiche la taille transférée, le temps et le débit.

6.2.2 Côté serveur

Quand un esclave reçoit ce **GET**, il :

1. vérifie la requête ;
2. fait un **stat()** sur le fichier demandé ;
3. ouvre le fichier ;
4. compare l'offset demandé avec la taille réelle ;
5. si l'offset est valable, se place au bon endroit ;
6. envoie une réponse **OK** ou **RESTART** ;
7. envoie ensuite le contenu du fichier par blocs.

6.2.3 Pourquoi la reprise marche

La reprise marche parce que :

- le client connaît la taille de son fichier local ;
- cette taille est envoyée dans **request_t.offset** ;
- le serveur renvoie l'offset accepté dans **response_t.offset** ;
- le client écrit ensuite au bon endroit dans le fichier local.

6.3 Commande `ls`

Quand nous faisons `ls`, le serveur :

1. parcourt le répertoire courant ;
2. ignore les noms cachés qui commencent par `.` ;
3. construit une grande chaîne avec un nom par ligne ;
4. envoie une réponse `OK` ;
5. envoie ensuite ce texte comme payload.

Le client lit ce texte et l’affiche tel quel.

6.4 Commande `put`

Quand nous faisons `put fichier`, le client :

1. vérifie le nom du fichier ;
2. fait un `stat()` local ;
3. ouvre le fichier à envoyer ;
4. met sa taille dans la requête ;
5. envoie la requête ;
6. attend un premier `OK` du serveur ;
7. envoie le contenu du fichier ;
8. attend un second `OK`.

Le serveur :

1. refuse si la session n’est pas authentifiée ;
2. envoie un premier `OK` s’il est prêt ;
3. reçoit le fichier et l’écrit localement ;
4. tente de le répliquer vers les autres esclaves ;
5. envoie enfin un second `OK` au client.

6.5 Commande `rm`

Quand nous faisons `rm fichier`, le serveur :

1. refuse si la session n’est pas authentifiée ;
2. supprime le fichier local ;
3. tente de propager la suppression aux autres esclaves ;
4. renvoie un statut final.

6.6 Commande `auth`

La commande `auth <login> <password>` agit sur la session courante.

Le serveur compare les identifiants reçus à deux constantes :

- login : `admin` ;
- mot de passe : `srftp`.

Si les identifiants sont bons, la session devient authentifiée. Sinon, elle ne l’est pas.

6.7 Commande `bye`

La commande `bye` est simple :

- le client envoie une requête `BYE` ;
- le serveur répond `OK` ;
- le client affiche `Bye!` ;
- la connexion se ferme.

7 Répartition de charge et réplication

7.1 Enregistrement des esclaves

Chaque esclave envoie au maître un message `slave_hello_t`. Ce message contient :

- la version du protocole ;
- l'identifiant de l'esclave ;
- son port client ;
- son port de contrôle ;
- son nom de machine.

Le maître vérifie ces informations, puis les garde dans un tableau local.

7.2 Carte du cluster

Quand tous les esclaves sont enregistrés, le maître construit un `slave_cluster_t` qui contient la liste complète des esclaves.

Chaque esclave reçoit cette carte. Grâce à elle, un esclave sait vers quels autres esclaves il doit envoyer une réplication.

7.3 Redirection du client

Quand un client se connecte au maître :

1. le maître choisit un esclave ;
2. il envoie au client le `slave_hello_t` de cet esclave ;
3. il ferme la connexion ;
4. le client ouvre ensuite sa vraie session avec cet esclave.

Le choix de l'esclave se fait en round-robin. C'est volontairement simple.

7.4 Reconnexion après panne

La reconnexion automatique existe seulement pour `get`.

Si le client détecte une erreur réseau pendant un `get`, il :

1. ferme la connexion cassée ;
2. tente de se reconnecter ;
3. si besoin, repasse par le maître ;
4. relance le même `get` ;
5. reprend le transfert grâce à l'offset local.

Dans la configuration actuelle :

- le nombre maximal d'essais vaut 5 ;
- le délai entre deux essais vaut 1 seconde.

7.5 Réplication de `put` et `rm`

Quand un esclave traite un `put` ou un `rm`, il essaie ensuite de prévenir tous les autres esclaves connus.

7.5.1 Pour put

L'esclave source :

1. ouvre une connexion vers le port de contrôle du pair ;
2. envoie une requête de réplication `FTP_REPL_PUT` ;
3. envoie aussi le contenu du fichier ;
4. attend un accusé de réception.

L'esclave cible reçoit le fichier, l'écrit localement, puis renvoie un statut.

7.5.2 Pour rm

L'esclave source :

1. ouvre une connexion vers le pair ;
2. envoie une requête `FTP_REPL_RM` ;
3. attend le statut final.

L'esclave cible supprime le fichier puis répond.

Attention

Notre réplication reste simple. Si la modification locale a réussi mais que la réplication échoue vers un autre esclave, nous écrivons une erreur dans les logs, mais nous ne revenons pas en arrière.

8 Explication par étapes du sujet

8.1 Étape 1 : base du service FTP

8.1.1 Q1 et Q2

Nous avons commencé par définir le langage du protocole :

- un type de requête avec GET, PUT, LS, RM, BYE, AUTH ;
- une structure de requête ;
- une structure de réponse.

Cela a donné une base commune au client et au serveur.

8.1.2 Q3 et Q4

Nous avons ensuite mis en place l'exécution :

- un client interactif ;
- un serveur qui écoute ;
- un pool de workers ;
- un arrêt propre sur SIGINT.

Aujourd'hui, NB_PROC vaut 1. Nous avons donc bien un mécanisme de workers, mais avec un seul worker par esclave dans la configuration actuelle.

8.1.3 Q5

Nous avons séparé le monde client du monde serveur :

- le client travaille dans `data_client/` ;
- les serveurs travaillent dans `data_server/`.

Le client ne lit donc pas directement les fichiers du serveur.

8.1.4 Q6 et Q7

Nous avons ensuite réalisé le premier vrai transfert :

- le client construit un GET ;
- le serveur ouvre le fichier demandé ;
- le serveur envoie une réponse puis le fichier ;
- le client écrit le fichier reçu ;
- le client affiche les statistiques.

8.2 Étape 2 : blocs, session longue et reprise

8.2.1 Q8

Nous avons envoyé les fichiers par blocs de taille 4096 octets. Cela évite de charger tout le fichier dans un seul gros buffer pour le transfert normal.

8.2.2 Q9

Nous avons gardé une boucle de session. Cela permet d'envoyer plusieurs commandes sur la même connexion TCP.

Par exemple, nous pouvons faire :

- `get a.txt` ;
- `get b.txt` ;
- `ls` ;
- `bye`.

8.2.3 Q10

Nous avons traité deux points :

- la reprise d'un téléchargement interrompu ;
- la robustesse si un client coupe la connexion pendant un envoi.

La reprise fonctionne grâce à l'offset. La robustesse fonctionne parce que les fonctions réseau importantes renvoient des erreurs au lieu de tuer tout le processus. Le worker peut donc terminer la session courante puis continuer à servir d'autres clients.

8.3 Étape 3 : maître et esclaves

8.3.1 Q11

Nous avons fixé les rôles et les ports :

- un maître sur 2121 ;
- deux esclaves ;
- des ports clients 3001 et 3002 ;
- des ports de contrôle 4001 et 4002.

8.3.2 Q12

Nous avons ajouté l'enregistrement des esclaves au démarrage. Le maître apprend qui sont les esclaves avant d'accepter les clients.

8.3.3 Q13

Nous avons ajouté la redirection :

- le client parle d'abord au maître ;
- le maître choisit un esclave ;
- le client parle ensuite à cet esclave.

8.3.4 Q14

Nous avons ajouté la reconnexion automatique pendant `get`. Si une connexion tombe, le client peut :

- se reconnecter ;
- recevoir une nouvelle redirection ;
- reprendre le téléchargement.

8.4 Étape 4 : commandes avancées

8.4.1 Q15

Nous avons ajouté `ls`. Le serveur envoie une liste texte des fichiers du répertoire courant.

8.4.2 Q16

Nous avons ajouté :

- `put` pour envoyer un fichier du client vers le serveur ;
- `rm` pour supprimer un fichier côté serveur ;
- la réplication simple de ces modifications vers les autres esclaves.

8.4.3 Q17

Nous avons ajouté une authentification de session. Sans authentification :

- `get` et `ls` restent autorisés ;
- `put` et `rm` sont refusés.

9 Fonctions importantes

9.1 Fonctions côté client

Fonction	Fichier	Rôle simple
<code>parse_command</code>	<code>src/clientFTP.c</code>	Analyse la ligne tapée par l'utilisateur.
<code>ftp_client_connect</code>	<code>src/clientFTP.c</code>	Se connecte soit au maître, soit directement à un esclave.
<code>ftp_client_reconnect</code>	<code>src/clientFTP.c</code>	Réessaie un <code>get</code> après une panne réseau.
<code>ftp_client_run</code>	<code>src/clientFTP.c</code>	Gère toute la boucle interactive du client.
<code>ftp_client_get</code>	<code>src/client_requests.c</code>	Envoie un <code>GET</code> , reçoit la réponse, reçoit le fichier et calcule les statistiques.
<code>ftp_client_put</code>	<code>src/client_requests.c</code>	Envoie un <code>PUT</code> en deux temps.
<code>ftp_client_ls</code>	<code>src/client_requests.c</code>	Envoie <code>LS</code> et reçoit le texte de la liste.
<code>ftp_client_rm</code>	<code>src/client_requests.c</code>	Envoie <code>RM</code> et lit le statut final.
<code>ftp_client_auth</code>	<code>src/client_requests.c</code>	Envoie les identifiants pour authentifier la session.
<code>ftp_client_bye</code>	<code>src/client_requests.c</code>	Termine proprement la session.

9.2 Fonctions côté serveur

Fonction	Fichier	Rôle simple
<code>ftp_server_run</code>	<code>src/serverFTP.c</code>	Démarre un esclave, ouvre les ports et lance les workers.
<code>register_slave_to_master</code>	<code>src/serverFTP.c</code>	Attend le maître puis envoie l'identité de l'esclave.
<code>worker_loop</code>	<code>src/serverFTP.c</code>	Attend un client et traite sa session.
<code>control_loop</code>	<code>src/serverFTP.c</code>	Attend les messages de réplication.
<code>ftp_handle_client</code>	<code>src/server_requests.c</code>	Boucle principale d'une session côté serveur.
<code>handle_get_request</code>	<code>src/server_requests.c</code>	Gère le téléchargement avec reprise.
<code>handle_put_request</code>	<code>src/server_requests.c</code>	Gère le téléversement et déclenche la réplication.
<code>handle_ls_request</code>	<code>src/server_requests.c</code>	Gère la liste des fichiers.
<code>handle_rm_request</code>	<code>src/server_requests.c</code>	Supprime un fichier puis tente de répliquer la suppression.
<code>handle_auth_request</code>	<code>src/server_requests.c</code>	Valide ou refuse l'authentification.
<code>ftp_handle_replication_connection</code>	<code>src/server_requests.c</code>	Traite un message de réplication reçu depuis un autre esclave.

9.3 Fonctions du protocole et du transfert

Fonction	Fichier	Rôle simple
<code>ftp_send_request / ftp_receive_request</code>	<code>src/ftp_protocol.c</code>	Envoient et lisent une requête complète au bon format réseau.
<code>ftp_send_response / ftp_receive_response</code>	<code>src/ftp_protocol.c</code>	Envoient et lisent une réponse complète au bon format réseau.
<code>ftp_send_slave_hello / ftp_receive_slave_hello</code>	<code>src/ftp_protocol.c</code>	Gèrent l'échange d'identité entre maître et esclaves.
<code>ftp_send_cluster_map / ftp_receive_cluster_map</code>	<code>src/ftp_protocol.c</code>	Gèrent l'envoi de la carte du cluster.
<code>ftp_send_replication_request / ftp_receive_replication_request</code>	<code>src/ftp_protocol.c</code>	Gèrent les messages de réplication.
<code>ftp_send_control_reply / ftp_receive_control_reply</code>	<code>src/ftp_protocol.c</code>	Gèrent les accusés de réception du canal de contrôle.
<code>ftp_send_fd_payload</code>	<code>src/ftp_transfer.c</code>	Envoie le contenu d'un fichier ouvert par blocs.
<code>ftp_receive_file_payload</code>	<code>src/ftp_transfer.c</code>	Reçoit un fichier et l'écrit sur disque, avec reprise si besoin.
<code>ftp_receive_buffer_payload</code>	<code>src/ftp_transfer.c</code>	Reçoit un texte, par exemple le résultat de <code>ls</code> .
<code>ftp_is_safe_filename</code>	<code>src/ftp_protocol.c</code>	Refuse les noms de fichiers dangereux.

9.4 Wrappers système utiles

Nous utilisons surtout quelques wrappers de `csapp` :

- `Open_listenfd` pour ouvrir une socket d'écoute ;
- `open_clientfd` pour ouvrir une connexion TCP ;
- `rio_readn` et `rio_writen` pour lire et écrire exactement le bon nombre d'octets ;
- `Fork` pour créer les workers ;
- `Signal` pour installer les handlers ;
- `Close` pour fermer proprement les descripteurs.

10 Tests

10.1 Scripts présents dans le dépôt

Le dépôt contient plusieurs scripts de test et de démonstration. Les deux plus importants pour la version finale sont :

- `scripts/demo_q1_q17.sh`;
- `scripts/test_questions_q1_q17.sh`.

10.2 Ce que ces scripts vérifient

Ces scripts vérifient notamment :

- la compilation du projet ;
- le démarrage du cluster maître / esclaves ;
- un `get` simple ;
- plusieurs commandes sur une seule session ;
- la reprise d'un fichier tronqué ;
- la robustesse après coupure brutale d'un client ;
- la redirection par le maître ;
- la reconnexion du client après panne d'un worker ;
- `ls`, `auth`, `put` et `rm` ;
- les traces de réplication dans les logs.

10.3 Méthodes de vérification

Les scripts utilisent des outils simples :

- `grep` pour chercher des messages dans les logs ;
- `test -f` pour vérifier qu'un fichier existe ;
- `sha256sum` pour comparer deux fichiers ;
- des attentes explicites dans les logs pour vérifier que le cluster est prêt.

10.4 Commandes utiles

Les commandes les plus utiles sont :

- `make all` pour construire le projet ;
- `make clean` pour nettoyer ;
- `make demo-q1-q17` pour lancer la démonstration complète ;
- `bash scripts/test_questions_q1_q17.sh` pour lancer la batterie de tests question par question.

11 Limites actuelles

Le projet répond au sujet, mais nous avons gardé quelques choix simples :

- les identifiants d'authentification sont codés en dur ;
- l'authentification vaut seulement pour la connexion TCP courante ;
- la reconnexion automatique existe seulement pour `get` ;
- le maître fait un round-robin simple, sans mesure réelle de charge ;
- la réplication est simple et sans journal persistant ;
- si un pair est absent pendant une réplication, nous n'avons pas de rattrapage différé ;
- les deux esclaves partagent le même dossier `data_server/` dans ce dépôt local ;
- la taille du pool vaut 1 par défaut, même si la structure du code accepte plusieurs workers.

12 Conclusion

Nous avons construit un mini service FTP complet pour le sujet.

Le projet montre plusieurs points importants :

- un protocole applicatif simple sur TCP ;
- des transferts de fichiers par blocs ;
- la reprise d'un téléchargement ;
- une architecture maître / esclaves ;
- une redirection client vers un esclave ;
- une reconnexion côté client ;
- des commandes avancées avec authentification et réplication simple ;
- une sérialisation explicite des messages réseau avec gestion de l'ordre des octets.

Nous avons volontairement gardé un design lisible. Ce n'est pas un FTP complet au sens industriel, mais c'est un projet cohérent, progressif et utile pour comprendre les bases des systèmes répartis et des réseaux.